Guía n.° 2.3.6

Actividad: Ejercicios de JavaFX con MVC, TableView, ObservableList, navegación entre vistas y validación de formularios

Aplicación práctica de organización por capas, gestión de tablas, cambio de vistas y control de entradas en una aplicación JavaFX

Guía n.°	2.2.8
Nombre de la guía	Actividad de ejercicios sobre MVC, TableView, ObservableList, navegación entre vistas y validación en JavaFX
Sigla	DSY1102
Asignatura	Desarrollo Orientado a Objetos
Experiencia de Aprendizaje	Aplicar los conceptos de organización MVC, manejo de datos con TableView y ObservableList, navegación entre vistas y validación de formularios en ejercicios guiados de JavaFX.
Tiempo	90 minutos
Modalidad de Trabajo	Individual o en parejas
Indicadores de Logro	Distingue responsabilidades del modelo, vista y controlador; propone tablas enlazadas a listas observables; organiza el cambio entre pantallas; y plantea validaciones coherentes para formularios de entrada.

Propósito de la guía

Desarrollar la capacidad de aplicar los conceptos trabajados en la guía anterior mediante ejercicios orientados al análisis, diseño y propuesta de soluciones en JavaFX. En esta actividad no se entregan respuestas resueltas: el estudiante deberá construir sus propias propuestas de estructura, código y justificación a partir de situaciones similares a un proyecto real.

Los ejercicios se organizan alrededor de una mini aplicación de gestión de estudiantes. A partir de ese contexto, se solicita identificar capas del patrón MVC, definir modelos, enlazar datos a una TableView, planificar navegación entre vistas y plantear validaciones de formularios antes de guardar información.

Instrucciones generales

- Lee cada situación antes de responder.
- Si el ejercicio pide código, escribe únicamente las partes necesarias para resolverlo.
- Usa nombres coherentes para clases, paquetes, vistas FXML, fx:id y métodos de evento.
- Justifica brevemente tus decisiones cuando el ejercicio lo requiera.
- Relaciona cada respuesta con el funcionamiento real de un proyecto JavaFX en IntelliJ IDEA.
- Si detectas posibles errores de diseño o configuración, anótalos como observación complementaria.

Formato sugerido de trabajo para el estudiante

1. Nombre del archivo, clase o componente implicado
2. Descripción breve de la solución propuesta
3. Código o pasos principales
4. Explicación del concepto aplicado
5. Observaciones o justificación de la decisión tomada

Caso base: Aplicación de gestión de estudiantes en JavaFX

Situación: Un estudiante está desarrollando en IntelliJ IDEA una pequeña aplicación JavaFX para registrar y visualizar estudiantes. El proyecto usa Maven, archivos FXML, controladores Java, una TableView enlazada a una ObservableList y dos pantallas: una vista principal con el listado y otra vista con un formulario de ingreso de datos.

Consigna general: A partir de este caso base, desarrolla los siguientes ejercicios proponiendo estructuras, explicaciones y fragmentos de código cuando corresponda.

Bloque 1: Organización MVC

Ejercicio 1: Identificación de capas del proyecto

Situación: El proyecto tiene clases Java, archivos FXML y una lista de estudiantes, pero el estudiante no sabe cómo separarlos correctamente.

Consigna: Explica cómo organizarías el proyecto siguiendo una idea básica del patrón MVC. Indica qué archivos o elementos ubicarías en modelo, vista y controlador.

Respuesta / propuesta	
Código o pasos	
Explicación	
Observaciones	

Ejercicio 2: Responsabilidad de cada capa

Situación: En una revisión, un compañero mezcla reglas de validación, carga de vistas y estructura de datos dentro de una sola clase.

Consigna: Describe la responsabilidad principal de modelo, vista y controlador dentro de una aplicación JavaFX como la trabajada en la guía.

Respuesta / propuesta	
Código o pasos	
Explicación	
Observaciones	

Bloque 2: Modelo y datos

Ejercicio 3: Diseño de la clase Estudiante

Situación: Se necesita representar un estudiante con datos mínimos para mostrarlo en una tabla.

Consigna: Propón una clase Estudiante indicando atributos, constructor y métodos de acceso necesarios para utilizarla en la aplicación.

Respuesta / propuesta	
Código o pasos	
Explicación	
Observaciones	

Ejercicio 4: Uso de ObservableList

Situación: El estudiante debe decidir cómo almacenar los registros para que la tabla se actualice automáticamente al agregar nuevos elementos.

Consigna: Explica por qué conviene usar una ObservableList en este proyecto y escribe una propuesta básica para declarar y devolver esa lista desde una clase repositorio o de apoyo.

Respuesta / propuesta	
Código o pasos	
Explicación	
Observaciones	

Bloque 3: TableView

Ejercicio 5: Diseño de la tabla principal

Situación: La vista principal debe mostrar los campos id, nombre, correo y edad.

Consigna: Propón la estructura general de una TableView en FXML indicando sus columnas y el propósito de cada una.

Respuesta / propuesta	
Código o pasos	
Explicación	
Observaciones	

Ejercicio 6: Vinculación de columnas con el modelo

Situación: La tabla ya existe en la interfaz, pero todavía no muestra los datos de la lista.

Consigna: Escribe una propuesta de controlador donde se configuren las columnas de la TableView y se vincule la tabla con la ObservableList de estudiantes.

Respuesta / propuesta	
Código o pasos	
Explicación	
Observaciones	

Ejercicio 7: Problemas frecuentes con la TableView

Situación: Al ejecutar la aplicación, la tabla aparece vacía aunque la lista observable sí contiene datos.

Consigna: Menciona al menos tres causas posibles de este problema y explica cómo revisarías cada una.

Respuesta / propuesta	
Código o pasos	
Explicación	
Observaciones	

Bloque 4: Navegación entre vistas

Ejercicio 8: Cambio desde la lista al formulario

Situación: Desde la vista principal se necesita abrir la pantalla de formulario al presionar un botón.

Consigna: Explica cómo usarías FXMLLoader y Scene para cargar otra vista FXML y mostrarla en el Stage principal.

Respuesta / propuesta	
Código o pasos	
Explicación	
Observaciones	

Ejercicio 9: Regreso a la vista principal

Situación: Después de guardar o cancelar, el usuario debe volver al listado principal.

Consigna: Propón la lógica que implementarías en el controlador del formulario para regresar a la vista principal.

Respuesta / propuesta	
Código o pasos	
Explicación	
Observaciones	

Ejercicio 10: Estructura de una clase de navegación

Situación: El estudiante quiere evitar repetir el mismo bloque de carga de vistas en varios controladores.

Consigna: Diseña una propuesta sencilla de clase de navegación que centralice la carga de vistas y el cambio de escenas.

Respuesta / propuesta	
Código o pasos	
Explicación	
Observaciones	

Bloque 5: Validación de formularios

Ejercicio 11: Reglas de validación del formulario

Situación: El formulario tiene campos de nombre, correo y edad.

Consigna: Define reglas básicas de validación para esos tres campos y escribe los mensajes que mostrarías al usuario cuando los datos sean incorrectos.

Respuesta / propuesta	
Código o pasos	
Explicación	
Observaciones	

Ejercicio 12: Propuesta de método guardarEstudiante

Situación: Se necesita un método que valide entradas y, si todo es correcto, agregue un nuevo estudiante a la lista observable.

Consigna: Escribe una propuesta de método guardarEstudiante que reciba datos desde los TextField, valide la información y agregue el registro cuando corresponda.

Respuesta / propuesta	
Código o pasos	
Explicación	
Observaciones	

Bloque 6: Integración del proyecto

Ejercicio 13: Diseño de una mini aplicación completa

Situación: Se desea construir una mini aplicación JavaFX de gestión de estudiantes o usuarios con dos vistas y una tabla principal.

Consigna: Propón la estructura completa del proyecto indicando nombre, paquetes, clases principales, vistas FXML, controladores, lista observable, navegación y validaciones que debería incluir.

Respuesta / propuesta	
Código o pasos	
Explicación	
Observaciones	

Criterios de evaluación sugeridos

Categoría	% logro	Descripción de niveles de logro
Comprensión de MVC	25%	Reconoce con claridad la función del modelo, la vista y el controlador dentro de una aplicación JavaFX.
Uso de TableView y ObservableList	25%	Plantea correctamente el enlace entre los datos y la tabla, con una organización coherente del modelo.
Navegación y validación	25%	Propone soluciones funcionales para cambiar de vistas y controlar entradas de formulario.
Claridad y justificación	25%	Presenta respuestas ordenadas, comprensibles y bien fundamentadas según el caso planteado.

Cierre

Al finalizar esta guía, el estudiante debería ser capaz de aplicar los conceptos de MVC, TableView, ObservableList, navegación entre vistas y validación de formularios dentro de una aplicación JavaFX más ordenada, clara y cercana a un proyecto real, proponiendo soluciones coherentes incluso antes de llegar a una implementación completa.